I Tried 100+ Claude Code Skills. These 6 Are The Best.

This guide breaks down the six core Claude Code skills (plus one bonus) that consistently solve real business problems by saving time, cutting costs, and removing mistakes. These are the skills behind the agents and automations clients actually pay for.

By: Nate Herk

I. The Core Mindset

Most online tutorials show off flashy, complex skills built for the sake of a cool video. Businesses don't pay for that. They pay for skills that are simple, even boring, but effective. The skills that consistently sell across industries (real estate, HVAC, coaching, marketing agencies, and more) all do at least one of three things:

1. 💰 **Save them money**
2. ⏱️ **Save them time**
3. ✅ **Remove human error**

Different industries face the same underlying problems, and these six skills keep showing up again and again. If you want to make money selling AI in 2026, don't overthink it. Start with what businesses are already paying for.

Quick terminology note: The word "skill" is used loosely throughout this guide. Technically, some of these are *plugins*. A skill is a markdown package that teaches Claude how to do a specific job better. A plugin is a larger package that can contain multiple skills plus extras like hooks or MCP servers. The distinction matters less than the outcome: each one makes Claude noticeably better at its job.

II. Skill #1: Skill Creator

The factory that builds every other skill.

Skill Creator is one of Anthropic's official skills. Instead of manually writing skill.md files, figuring out formatting, and testing trigger conditions, you describe what you want in plain English.

Claude drafts the skill, tests it, iterates on it, and packages it into something reusable. You can even drop in an SOP and Claude will turn it into a working skill.

Why It Matters

Most people who get into Claude Code try to write skills manually and immediately hit a wall. They don't know the structure or what makes a skill reliable versus flaky. They either give up or cobble something together that breaks the second a client touches it.

Real-World Example

A real estate agency wastes hours every week writing property descriptions. Without Skill Creator, you'd be writing the entire skill.md file by hand, configuring trigger conditions, and debugging through multiple iterations before getting something stable. With Skill Creator, you explain it conversationally and walk away with a tested, packaged skill.

Why It Sells (Indirectly)

Clients don't pay for Skill Creator itself. But every skill you sell to an HVAC company, real estate agency, or coach comes out of this one. It's the factory that builds the product, which means indirectly, it makes every other dollar possible.

Installation

```
None
/plugin install skill-creator@claude-plugins-official
```

Pro tip: Install this globally on a user scope so it auto-invokes across every project without you having to think about it.

III. Skill #2: Superpowers

Forces Claude to work the way a senior developer works.

Instead of jumping in and writing code right away, Superpowers makes Claude:

1. Step back and plan the whole thing first
2. Work in an isolated environment so nothing breaks the main project
3. Write tests before writing code
4. Brainstorm and review its own work in two stages: once for spec match, once for code quality

Why It Matters

The number one failure mode in Claude Code is rushed or sloppy code. You ask for something, Claude sprints to write it, and on the surface it looks fine. Then when you (or your client) actually run it, it falls apart. Superpowers slows Claude down just enough to actually think the problem through and prevents skipping steps.

Real-World Example

An HVAC company paying for a dispatch system or a marketing agency paying for a custom reporting tool doesn't care how many lines of code were written. They care that the system works when their business depends on it. Superpowers helps Claude plan, write tests, and catch edge case scenarios before the client ever sees the build.

The Realistic Win

Superpowers won't one-shot every project. A large part of building is QA, but getting to roughly 80% on the first pass instead of 60% is a major win. Fewer debugging cycles means less time spent fixing things and lower token costs. With over 150,000 stars on GitHub, it's one of the most popular community skills available.

Installation

```
None
/plugin install superpowers@claude-plugins-official
```

IV. Skill #3: GSD

Solves context rot by giving Claude a clean working environment.

If Superpowers controls *how* Claude writes code, GSD controls *the environment* Claude writes it in.

The Problem It Fixes

Every Claude Code user eventually hits the wall: a session starts strong, then somewhere around halfway through the context window, Claude starts to slip. Code gets sloppy, requirements get forgotten, steps get skipped, and Claude might even claim things are done when they aren't. This is called **context rot**.

How It Works

GSD spawns fresh sub-agents for each task, and each one has a clean context window focused only on what it needs to know. Your main session stays clean, and every task gets the full power of Claude's context window instead of the leftovers.

Built-In Quality Gates

- **Scope reduction detection:** Catches the planner when it silently drops a requirement you asked for
- **Security enforcement:** Anchors verification to your threat model
- **Autonomous mode:** Hand Claude a spec and walk away. It plans, executes, commits, and works through the whole project without you having to babysit it

These aren't things you'd manually think to check, especially without a software engineering background, which is exactly why automating them matters.

Token Honesty

GSD isn't a tokens-saving plugin. Sub-agents cost tokens. What it actually saves is the hours you'd otherwise spend redoing work that Claude broke because it forgot what you asked in the first place.

How It Pairs With Superpowers

- **Superpowers** gives Claude the **process** (plan, test, review, repeat)
- **GSD** gives Claude the **clean context** so the process actually works every time

Installation

```
None
npx get-shit-done-cc --claude --global
```

Type `/gsd-help` inside Claude Code to see the full list of available commands.

V. Skill #4: /review and /ultra-review (Built-In)

Already inside Claude Code. Most people just aren't using it.

These aren't installed separately. They ship with Claude Code, and they're powerful and underused.

/review

A structured local code review on what you just built. Looks for bugs, edge cases, and design issues, all the stuff a careful reviewer would flag. Runs locally, fast, and costs nothing beyond your usual usage tokens.

/ultra-review

Launched alongside Opus 4.7. Instead of running locally, it uploads your branch to a cloud sandbox and spins up a fleet of reviewer agents in parallel. Each agent attacks your codebase from a different angle: logic, security, performance, edge cases.

The kicker: before any bug shows up on your list, it has to be **independently reproduced and verified**. You won't get a pile of style nitpicks or false positives. You get confirmed bugs.

Where They Fit in the Workflow

1. Plan with Superpowers
2. Execute with clean context using GSD
3. Run `/review` for fast feedback on everything
4. Run `/ultra-review` before merging anything important (big refactors, payments, auth, database migrations)

Requirements for /ultra-review

- Claude Code version **2.1.86 or later**
- Signed in with a Claude account (an API key alone won't work)
- Runs in the background, takes 10 to 20 minutes

Cost

Pro and Max plans get 3 free runs to try. After that, expect roughly $5 to $20 per run depending on size. Worth it when a production bug costs more than the run.

Installation

No install required. Just type `/review` or `/ultra-review` in your terminal on Claude Code 2.1.86 or later.

VI. Skill #5: Context Mode

Keeps the garbage out, and remembers where you left off.

The Problem

Every tool call dumps raw data into your context window:

- A Playwright snapshot might be 56 KB
- 20 GitHub issues might be 59 KB

After roughly 30 minutes of real work, **40% of your context window is just garbage**: log output, raw data, stuff Claude never actually needs to look at. When Claude runs out of space and compacts the conversation, it forgets what files it was editing, what tasks were in progress, and what you last asked it to do.

What Context Mode Fixes

1. Keeps the garbage out. Tool calls and URL fetches route through a sandbox. Your code runs in an isolated subprocess, raw output gets captured, and only the part Claude actually needs comes back into context.

Their published benchmarks:

- 56 KB Playwright snapshot becomes **299 bytes**
- 46 KB access log becomes **155 bytes**
- 315 KB total raw output across a session becomes **5 KB**

Run `/context-mode:ctx-stats` at any point to see your own numbers.

2. Tracks every meaningful event in a local SQL database. Every file edit, every task created, every decision made, every error that comes up. When Claude compacts the conversation, Context Mode rebuilds a session snapshot and injects it back in. The model picks up exactly where you left off, with files, tasks, and last prompt intact.

The Practical Result

Sessions that used to fall apart around the 30-minute mark now run for three hours without slowdown. You stop wasting prompts reminding Claude of context it already had.

Installation

```
None
/plugin marketplace add mksglu/context-mode
/plugin install context-mode@context-mode
```

Restart Claude Code. The plugin auto-installs the MCP server, hooks, and routing instructions.

VII. Skill #6: Claude Mem

Carries knowledge across all of your future sessions.

If Context Mode keeps the *current* session clean, Claude Mem makes sure knowledge survives *between* sessions.

The Problem

Claude Code starts every new session from scratch. You explain the project for the fifth time this month. Every new conversation costs roughly 10 minutes and thousands of tokens just catching Claude up.

CLAUDE.md and memory files help, but you have to maintain them by hand. Miss an update, lose the context.

How It Works

Claude Mem hooks into Claude's session lifecycle and automatically captures:

- File edits
- Decisions
- Bug fixes
- Commands

It uses Claude's own Agent SDK to compress everything into semantic summaries, then stores it in a local SQLite database with vector search. New sessions get the relevant pieces auto-injected. It also auto-generates folder-level CLAUDE.md files and updates them as you work, so your project documentation literally writes itself.

The Token Savings

Instead of dumping every past observation into context, Claude Mem uses a three-layer search system:

1. Returns a compact index of observations
2. Pulls a timeline around the ones that matter
3. Fetches full details only for the handoff you need

The repo reports about **10x token savings on retrieval** compared to dumping everything in at session start.

The Practical Result

- No startup tax on every new session

- No repeating yourself
- Pick up a project from two weeks ago and Claude already knows where you left off and what you've decided
- A local web viewer lets you actually see what Claude remembers about your project

Installation

```
None
/plugin marketplace add thedotmack/claude-mem
/plugin install claude-mem
```

⚠️ **Warning from the repo:** Don't run the npm install command. That installs the SDK library only, the hooks never register, and nothing actually works. Stick with the two commands above.

VIII. Bonus: Frontend Design Skill

Stops your Claude Code designs from looking AI-generated.

This is Anthropic's official front-end design skill. Install it globally so every time you build websites, slide decks, or design-driven artifacts inside Claude Code, the output looks polished instead of obviously AI.

When to Use It

- Designing inside Claude Code directly
- Bringing a Claude Design (Anthropic's AI-powered design product) project back into Claude Code for further development

Installation

```
None
/plugin install frontend-design@claude-plugins-official
```

Install globally so it's always available across projects.

IX. How to Actually Sell These Skills

You don't sell the skill. You sell the outcome.

What Clients Actually Buy

- Saving the business owner 10 hours a week
- Cutting admin mistakes and human error
- Speeding up lead response time, or generating more leads
- Helping them focus on profit-driving activities

The Reframe

Stop offering to "build an AI workflow." Start offering to save time, cut costs, and help businesses focus on making more profit.

A Simple Starting Path

1. **Pick one skill.** Don't try to use all six right away.
2. **Learn it deeply.** Build a few workflows with it.
3. **Demo it for a business owner.** They won't care about your experience. They'll care about the value of what you're showing them.
4. **Compound.** As you understand how the skills connect, you build better automations faster and cheaper. That means more demos, more content, and more parts of your own business automated.

The deeper your fluency, the better you communicate value to business owners.

X. Quick Reference: All Installation Commands

Skill	Role	Command
Skill Creator	Builds every other skill	`/plugin install skill-creator@claude-plugins-official`
Superpowers	Forces senior-dev workflow	`/plugin install superpowers@claude-plugins-official`
GSD	Clean context per task	`npx get-shit-done-cc --claude --global`
/review and /ultra-review	Verified bug-finding	Built-in (Claude Code 2.1.86+)

Context Mode	Strips garbage, restores sessions	`/plugin marketplace add mksglu/context-mode` `/plugin install context-mode@context-mode`
Claude Mem	Cross-session memory	`/plugin marketplace add thedotmack/claude-mem` `/plugin install claude-mem`
Frontend Design *(bonus)*	Polished, non-AI-looking design	`/plugin install frontend-design@claude-plugins-official`

Want to connect with others building and monetizing AI automation?

Become an AIS Plus Member